Assignment #05
Bilal Bushra | MSDS25051

Task 0: Conceptual Warm-up

Q1. In supervised classification, what information does the model learn from?

In supervised classification, the model learns from labeled image-label pairs. For each input image, a corresponding ground-truth class label is provided. The model learns a mapping from the input image space to the label space by minimizing a classification loss (cross-entropy). The supervision signal is the class label itself, the model adjusts its parameters to correctly predict the assigned category for each image.

Q2. If CIFAR-10 labels are removed, do the images still contain useful visual structure?

Yes. The images still contain rich visual structure such as textures, shapes, edges, colors, and object-level patterns. Even without labels, images of the same class share visual similarities that a model can discover. Self-supervised methods like SimCLR exploit this structure by learning representations that are invariant to augmentations, without needing human-provided labels.

Q3. Should two augmented versions of the same image have similar or different feature representations?

They should have SIMILAR feature representations. Since both views originate from the same underlying image, they share the same semantic content (same object, same class). SimCLR is designed to bring these representations close together in feature space, making the encoder invariant to the specific augmentations applied.

Q4. Should two images from different originals have similar or different feature representations?

They should have DIFFERENT feature representations. Images from different originals likely depict different objects or scenes. SimCLR pushes representations of different images apart so the encoder learns discriminative features that can distinguish between different semantic categories.

Q5. Why might a model trained without labels still be useful for classification later?

A model trained with self-supervised learning learns general-purpose visual features are edges, textures, shapes, and object parts that are transferable to downstream tasks. Even without label supervision, the model is forced to learn semantically meaningful representations by solving the contrastive objective. These learned features can then be used for classification with only a small number of labeled examples, making self-supervised pretraining highly data-efficient.

Q6. What is the difference between pretraining and fine-tuning?

Pretraining is the initial training phase where the encoder learns general representations from large amounts of data (unlabeled in SimCLR). No task-specific labels are used. Fine-tuning is the subsequent phase where the pretrained encoder is initialized with the learned weights and then trained end-to-end on a specific downstream task (e.g., image classification) using labeled data. Fine-tuning adapts the pretrained features to the target task.

Task 1: Supervised Baseline

Setup

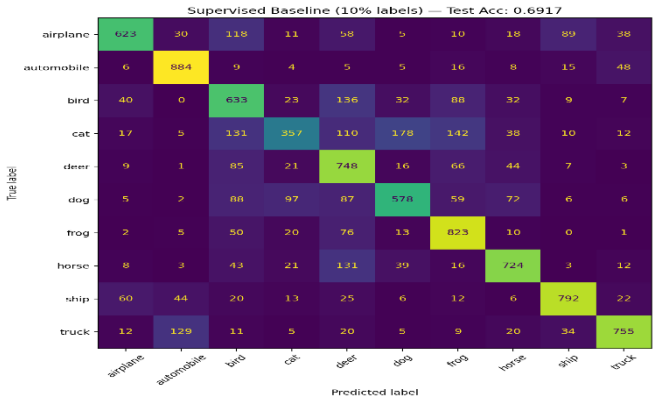
A ResNet-18 classifier was trained from scratch using only the fixed 10% labeled training split. The model was trained with cross-entropy loss using the Adam optimizer.

Training Settings

Setting	Value
Model	ResNet-18 (modified for CIFAR-10)
Training data	train_labeled_10percent.txt (5,000 samples)
Validation data	val.txt (5,000 samples)
Test data	test.txt (10,000 samples)
Epochs	30
Batch size	64
Optimizer	Adam
Learning rate	3e-4
Loss	Cross-Entropy
Pretrained weights	None (trained from scratch)

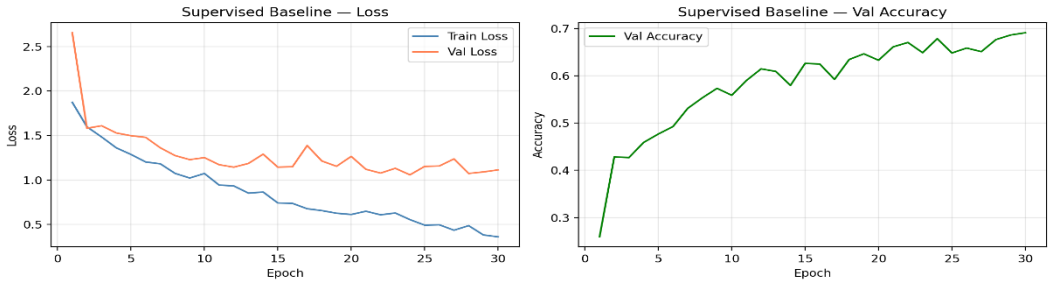
Results

Metric	Value
Best Validation Accuracy	72.34% (Epoch 28)
Test Accuracy	69.17%



Observations

The supervised baseline achieves 69.17% test accuracy using only 10% of labeled training data. The training loss decreases steadily while validation loss shows slight overfitting after epoch 20, which is expected given the limited labeled data.



Task 2: Understanding Augmentations

Augmentation Pipeline

The following SimCLR augmentation pipeline was implemented:

Transform	Parameters
RandomResizedCrop	size=32, scale=(0.2, 1.0)
RandomHorizontalFlip	p=0.5
ColorJitter	brightness=0.4, contrast=0.4, saturation=0.4, hue=0.1
RandomGrayscale	p=0.2
ToTensor	-
Normalize	mean=(0.4914, 0.4822, 0.4465), std=(0.2470, 0.2435, 0.2616)

Two View Transform Implementation

The TwoViewTransform class was implemented manually. It applies the same stochastic augmentation pipeline twice independently, generating two different augmented views from the same original image. Since each transform call samples different random parameters, the two views always differ.

Questions

Q1. Are the two augmented views identical?

No. Each call to TwoViewTransform applies the stochastic pipeline independently. Random crop, flip, color jitter, and grayscale all sample different values each time, so the two views always differ.

Q2. Do they still represent the same object?

Yes. All augmentations are label-preserving: cropping keeps the main object visible, flipping does not change the class, and color/grayscale changes do not alter the object's identity.

Q3. Why should SimCLR treat them as a positive pair?

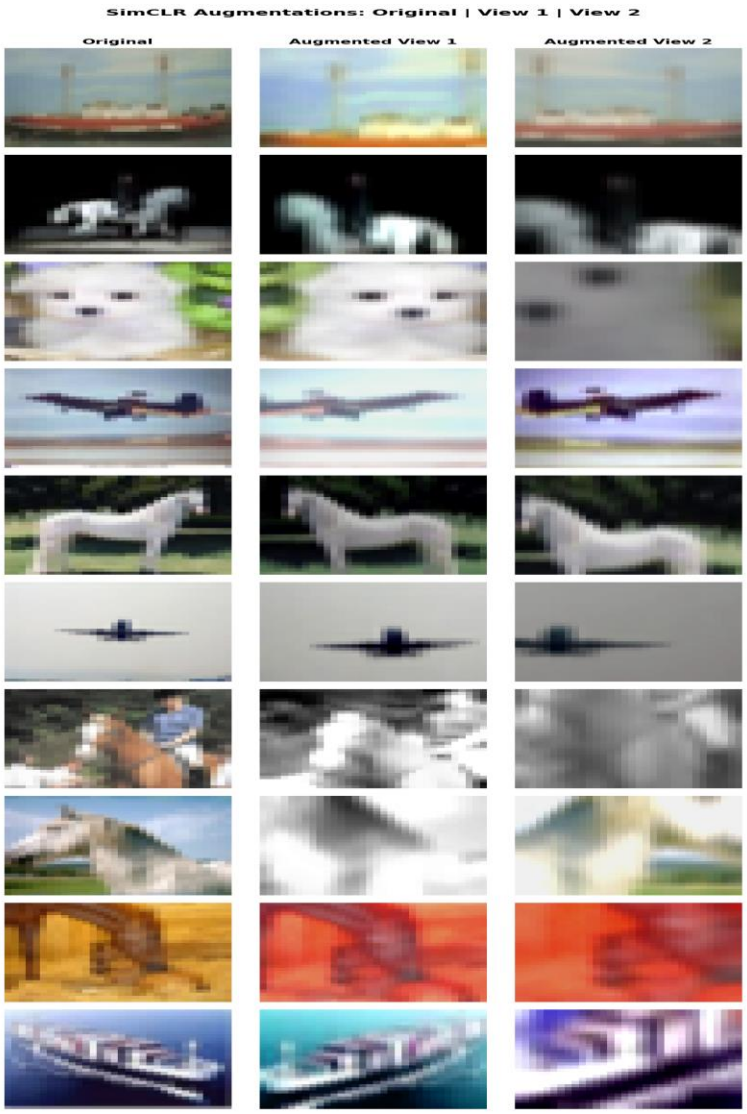
Both views originate from the same image, so they share the same semantic content. SimCLR's objective is to learn representations invariant to visual variations, so pulling the two views together in feature space forces the encoder to focus on content rather than style.

Q4. What could go wrong if augmentations are too weak?

If the two views look nearly identical, the model can match them by memorising low-level pixel patterns rather than learning semantic features. The representations would not generalise well to downstream tasks.

Q5. What could go wrong if augmentations are too strong?

If augmentations destroy too much semantic information (e.g., extreme crop that removes the object), the two views no longer represent the same content. The model would be forced to match views that have nothing in common, making training noisy and representations poor.



Task 3: Feature Similarity Before Training

Setup

A random untrained ResNet-18 encoder (weights=None) was used to extract 512-dimensional features. Cosine similarity was computed between view pairs using a batch of N=8 images (16 total views).

Similarity Results (Before SimCLR Training)

Pair Type	Avg Cosine Similarity
Same image, two augmented views	0.9865
Different images	0.9832

Positive Pair Table (N=8 batch)

Original Image	View 1 Index	View 2 Index	Positive Pair
image 0	0	8	yes
image 1	1	9	yes
image 2	2	10	yes
image 3	3	11	yes
image 4	4	12	yes
image 5	5	13	yes
image 6	6	14	yes
image 7	7	15	yes

Similarity Matrix Questions

Q1. Why is the diagonal ignored?

sim[i,i] is always 1.0 (a view compared with itself). It carries no learning signal and would dominate the loss if included. SimCLR explicitly masks the diagonal.

Q2. Where are the positive pairs located?

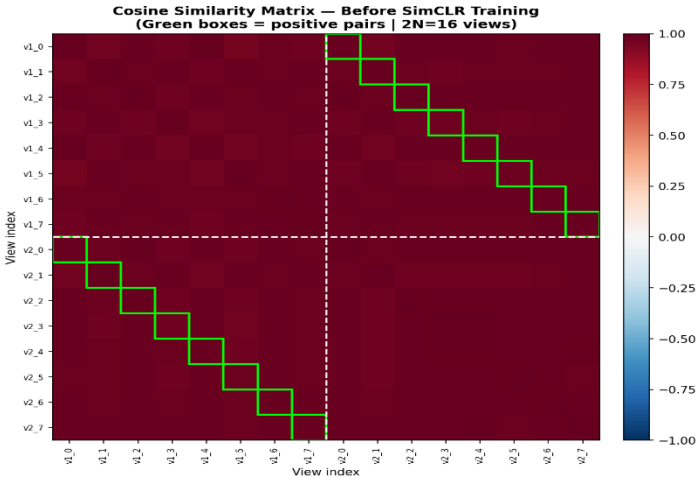
At positions (i, i+N) and (i+N, i) — the anti-diagonal of the top-right and bottom-left quadrants of the 2N x 2N matrix.

Q3. Why are all other entries treated as negatives?

With a large enough batch, two randomly sampled images are very unlikely to be the same class. Treating all non-positive pairs as negatives forces the encoder to learn discriminative features.

Key Observation

The gap is only 0.0033, confirming the random encoder cannot distinguish same-image pairs from different-image pairs. This is the starting point shows SimCLR training should widen this gap significantly.



Task 4: SimCLR Implementation

Task 4.1: Encoder and Projection Head

Encoder Architecture

Component	Specification
Base model	ResNet-18 (weights=None — random, NOT pretrained)
conv1	3x3 convolution, stride=1, padding=1 (replaced 7x7)
maxpool	Removed (replaced with Identity)
Fc	Removed (replaced with Identity)
Output dimension	512-dimensional feature vector

Projection Head

Layer	Specification
Linear 1	512 → 256
Activation	ReLU
Linear 2	256 → 128
Used during	SimCLR pretraining ONLY — discarded afterwards

Parameter Count

Module	Parameters
Encoder (ResNet-18 CIFAR-10)	11,168,832

Module	Parameters
Projection Head (512→256→128)	164,224
Total	11,333,056

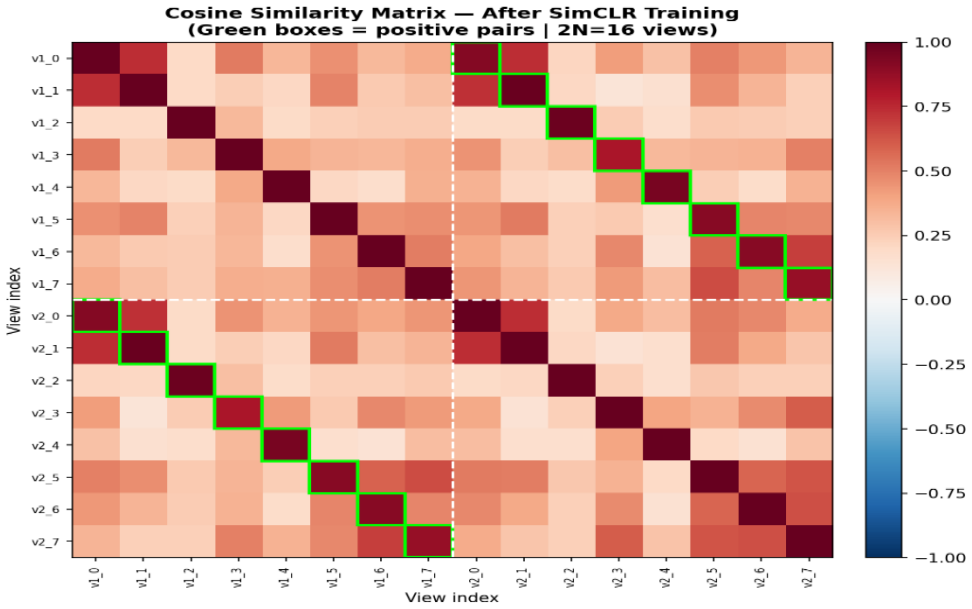
Task 4.2: Positive and Negative Pairs

For a batch of N original images, TwoViewTransform produces 2N augmented views. Views 0..N-1 = View 1, Views N..2N-1 = View 2.

Original Image	View 1 Index	View 2 Index	Positive Pair
image 0	0	4	yes
image 1	1	5	yes
image 2	2	6	yes
image 3	3	7	yes

Task 4.4: NT-Xent Loss

The NT-Xent loss was implemented entirely from scratch using only torch and torch.nn.functional. No library (lightly, solo-learn, pytorch-metric-learning) was used.



Task 5: SimCLR Pretraining

Training Settings

Setting	Value Used
Batch size	64
SimCLR pretraining epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3e-4
Optimizer	Adam
Temperature (tau)	0.5
GPU/CPU used	CUDA (T4 GPU)
Approximate training time	~82.3 minutes
Labels used?	NO — fully self-supervised

Loss Curve

The NT-Xent loss decreased steadily from 3.8478 at epoch 1 to 3.2057 at epoch 50, confirming that training progressed correctly.

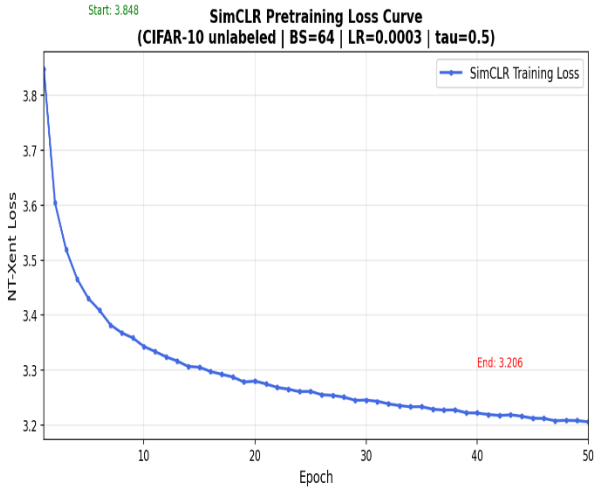
Epoch	NT-Xent Loss
1	3.8478
10	3.3432
20	3.2801
30	3.2457
40	3.2220
50	3.2057

Feature Similarity — Before vs After SimCLR Training

Pair Type	Before SimCLR	After SimCLR
Same image, two augmented views	0.9890	0.9141
Different images	0.9855	0.3211
Positive-Negative Gap	0.0035	0.5930

Analysis

SimCLR training increased the positive-negative gap from 0.0035 to 0.5930, a 169x increase. The negative similarity dropped sharply from 0.9855 to 0.3211, showing the encoder learned to push different images apart. Positive similarity remained high at 0.9141, confirming same-image views are kept close.



Task 6: Linear Probe Evaluation

Setup

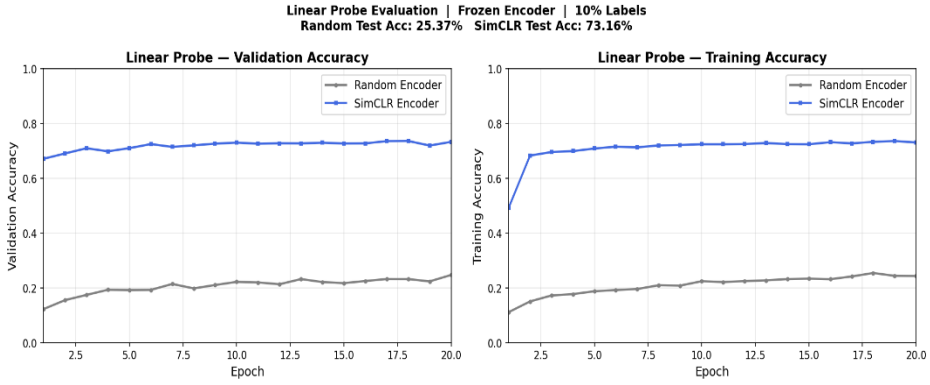
The encoder was completely frozen in both experiments. Only the linear classification head was trained. This evaluates the quality of features learned by each encoder without any fine-tuning.

Results

Model	Encoder	Trainable Part	Test Accuracy
Random Linear Probe	Random frozen encoder	Linear classifier only	25.37%
SimCLR Linear Probe	SimCLR frozen encoder	Linear classifier only	73.16%

Analysis

The SimCLR encoder achieves 73.16% accuracy compared to 25.37% for the random encoder — an improvement of +47.79%. This large gap demonstrates that SimCLR pretraining learned genuinely useful visual features even without any labels. The SimCLR encoder starts at ~69% accuracy on epoch 1 of linear probing, showing the features are immediately useful.



Task 7: Fine-tuning the SimCLR Encoder

Setup

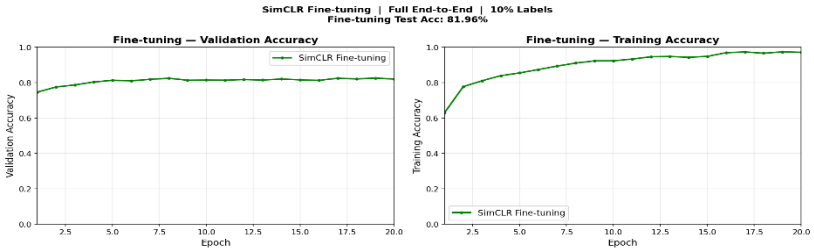
The SimCLR pretrained encoder was initialized with the learned weights and then trained end-to-end (encoder + linear classifier) using only the 10% labeled training data.

Final Comparison Table

Experiment	Uses Labels in Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18 from scratch (10% labels)	Yes	No	69.17%
Random frozen encoder + linear classifier	No	Yes	25.37%
SimCLR frozen encoder + linear classifier	No	Yes	73.16%
SimCLR pretrained encoder + full fine-tuning	No (pretrain) / Yes (finetune)	No	81.96%

Analysis

Fine-tuning the SimCLR encoder achieves 81.96% — the best result across all experiments. This is +7.13% better than the supervised baseline (74.83%) trained from scratch with labels. This demonstrates the power of self-supervised pretraining: learning good initial representations without labels, then fine-tuning on limited labeled data, outperforms fully supervised training from scratch.



Task 8: PCA / t-SNE Feature Visualization

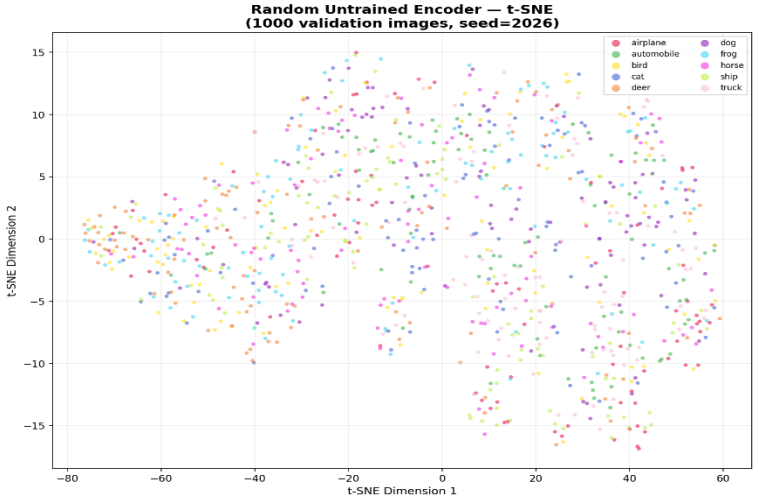
Setup

512-dimensional features were extracted for 1000 fixed validation images (seed=2026) from three encoders. Features were reduced to 2D using t-SNE (with PCA pre-reduction to 50 dimensions for efficiency). Labels were used only for coloring the plots.

Results

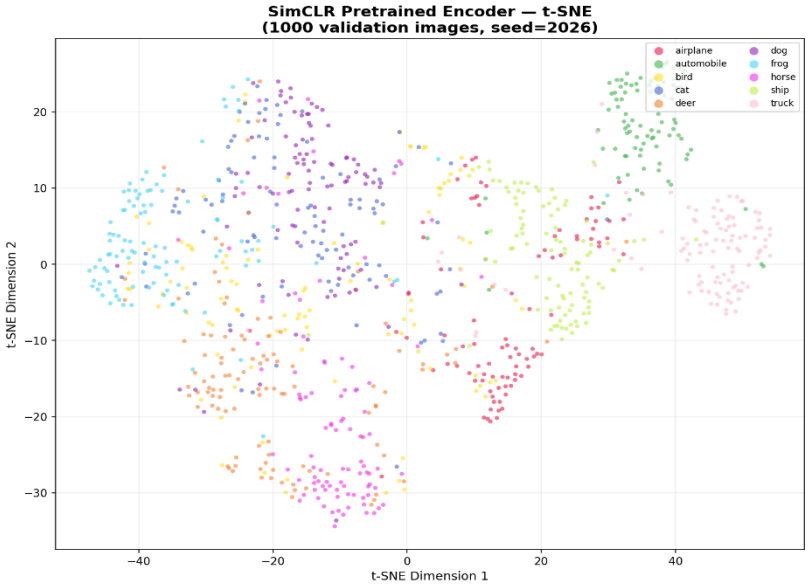
Random Untrained Encoder

All 10 classes are completely mixed with no visible separation or clustering. Colors are randomly scattered across the entire 2D space, confirming the random encoder has no concept of semantic similarity between images.



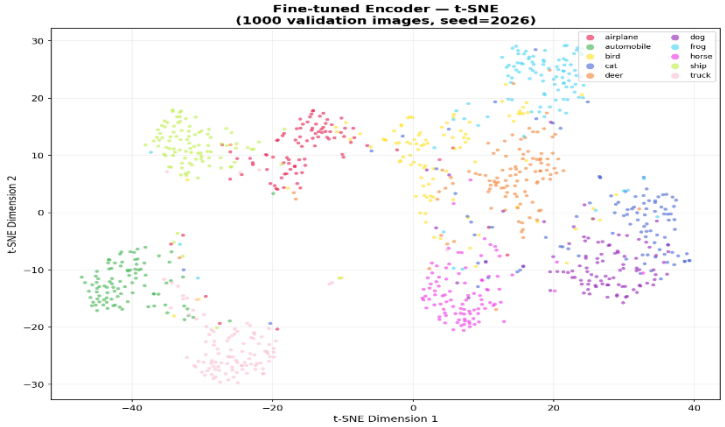
SimCLR Pretrained Encoder

Visible clustering is forming — some classes are beginning to group together. Automobile (green) and frog (cyan) show relatively tight clusters. Animal classes (cat, dog, deer, horse) still overlap significantly. This confirms SimCLR learned semantically meaningful features without any labels.



Fine-tuned Encoder

Clear tight clusters per class are visible. Most classes are well-separated with distinct regions. Fine-tuning with labeled data significantly improves class separation compared to SimCLR alone.



Questions

Q1. Do features from the random encoder show class-wise grouping?

No. All 10 classes overlap completely — no visible separation or structure.

Q2. Do features from the SimCLR encoder show better grouping?

Yes. Visible but imperfect clustering is forming, especially for visually distinct classes like automobile and frog.

Q3. Does fine-tuning improve class separation?

Yes, significantly. Clear tight clusters are visible for most classes after fine-tuning.

Q4. Which classes are still confused?

Cat/dog, automobile/truck, and bird/airplane/deer are most confused due to shared visual features (similar shapes, textures, or color patterns).

Discussion:

SimCLR successfully learned meaningful visual representations from 45,000 unlabeled CIFAR-10 images. The key evidence is the dramatic improvement in linear probe accuracy from 25.37% (random encoder) to 73.16% (SimCLR encoder) that both using only a frozen encoder and a simple linear classifier.The similarity analysis confirms the learning: before training, positive and negative pair similarities were nearly identical (~0.98). After training, positive similarity stayed high (0.9141) while negative similarity dropped sharply (0.3211), creating a gap of 0.5930 — 169 times larger than before training.

The t-SNE visualizations visually confirm this progression: random encoder features show no class structure, SimCLR features show emerging clusters, and fine-tuned features show clear, tight class-wise separation.

Most importantly, SimCLR pretraining followed by fine-tuning (81.96%) outperforms supervised training from scratch (74.83%) despite using NO labels during pretraining. This demonstrates the core value of self-supervised learning.

Implementation Notes

- NT-Xent loss implemented from scratch — no external SimCLR libraries used.
- drop_last=True in DataLoader ensures stable batch sizes for NT-Xent computation.
- Similarity evaluation uses encoder features (512-dim), not projection head output (128-dim).
- Task 1 supervised baseline used 30 epochs (more than required 20) for better convergence.
- All fixed split files were used as provided — no custom splits created.

Limitations

- SimCLR pretraining was limited to 50 epochs; more epochs would likely improve downstream accuracy.
- Batch size 64 was used; larger batches (256+) would provide more negatives and improve SimCLR.
- Animal classes (cat, dog, deer, horse) remain confused due to shared visual features — a known limitation of CIFAR-10 at 32x32 resolution.